

openai-java — README

OpenAI Java API Library



maven-central v4.30.0

javadoc 4.30.0

The OpenAI Java SDK provides convenient access to the [OpenAI REST API](#) from applications written in Java.

The REST API documentation can be found on platform.openai.com. Javadocs are available on javadoc.io.

Installation



Try `openai-java-spring-boot-starter` if you're using Spring Boot!

Gradle



```
implementation("com.openai:openai-java:4.30.0")
```

Maven



```
<dependency>
  <groupId>com.openai</groupId>
  <artifactId>openai-java</artifactId>
  <version>4.30.0</version>
</dependency>
```

Requirements



This library requires Java 8 or later.

Usage



💡 Tip

See the [openai-java-example](#) directory for complete and runnable examples!

The primary API for interacting with OpenAI models is the [Responses API](#). You can generate text from the model with the code below.

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.ChatModel;
import com.openai.models.responses.Response;
import com.openai.models.responses.ResponseCreateParams;

// Configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID` and `OPENAI_PROJECT_ID` environment
variables
OpenAIClient client = OpenAIOkHttpClient.fromEnv();

ResponseCreateParams params = ResponseCreateParams.builder()
    .input("Say this is a test")
    .model(ChatModel.GPT_5_2)
    .build();
Response response = client.responses().create(params);
```

The previous standard (supported indefinitely) for generating text is the [Chat Completions API](#). You can use that API to generate text from the model with the code below.

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

// Configures using the `openai.apiKey`, `openai.orgId`, `openai.projectId`,
`openai.webhookSecret` and `openai.baseUrl` system properties
// Or configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID`, `OPENAI_PROJECT_ID`,
`OPENAI_WEBHOOK_SECRET` and `OPENAI_BASE_URL` environment variables
OpenAIClient client = OpenAIOkHttpClient.fromEnv();

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .addUserMessage("Say this is a test")
    .model(ChatModel.GPT_5_2)
    .build();
ChatCompletion chatCompletion = client.chat().completions().create(params);
```

Client configuration



Configure the client using system properties or environment variables:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;

// Configures using the `openai.apiKey`, `openai.orgId`, `openai.projectId`,
`openai.webhookSecret` and `openai.baseUrl` system properties
// Or configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID`, `OPENAI_PROJECT_ID`,
`OPENAI_WEBHOOK_SECRET` and `OPENAI_BASE_URL` environment variables
OpenAIClient client = OpenAIOkHttpClient.fromEnv();
```

Or manually:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;

OpenAIClient client = OpenAIOkHttpClient.builder()
    .apiKey("My API Key")
    .build();
```

Or using a combination of the two approaches:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;

OpenAIClient client = OpenAIOkHttpClient.builder()
    // Configures using the `openai.apiKey`, `openai.orgId`, `openai.projectId`,
`openai.webhookSecret` and `openai.baseUrl` system properties
    // Or configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID`, `OPENAI_PROJECT_ID`,
`OPENAI_WEBHOOK_SECRET` and `OPENAI_BASE_URL` environment variables
    .fromEnv()
    .apiKey("My API Key")
    .build();
```

See this table for the available options:

Setter	System property	Environment variable	Required	Default value
<code>apiKey</code>	<code>openai.apiKey</code>	<code>OPENAI_API_KEY</code>	true	-
<code>organization</code>	<code>openai.orgId</code>	<code>OPENAI_ORG_ID</code>	false	-
<code>project</code>	<code>openai.projectId</code>	<code>OPENAI_PROJECT_ID</code>	false	-
<code>webhookSecret</code>	<code>openai.webhookSecret</code>	<code>OPENAI_WEBHOOK_SECRET</code>	false	-
<code>baseUrl</code>	<code>openai.baseUrl</code>	<code>OPENAI_BASE_URL</code>	true	<code>"https://api.openai.com/v1"</code>

System properties take precedence over environment variables.

 Tip

Don't create more than one client in the same application. Each client has a connection pool and thread pools, which are more efficient to share between requests.

Modifying configuration



To temporarily use a modified client configuration, while reusing the same connection and thread pools, call `withOptions()` on any client or service:

```
import com.openai.client.OpenAIClient;

OpenAIClient clientWithOptions = client.withOptions(optionsBuilder -> {
    optionsBuilder.baseUrl("https://example.com");
    optionsBuilder.maxRetries(42);
});
```

The `withOptions()` method does not affect the original client or service.

Requests and responses



To send a request to the OpenAI API, build an instance of some `Params` class and pass it to the corresponding client method. When the response is received, it will be deserialized into an instance of a Java class.

For example, `client.chat().completions().create(...)` should be called with an instance of `ChatCompletionCreateParams`, and it will return an instance of `ChatCompletion`.

Immutability



Each class in the SDK has an associated **builder** or factory method for constructing it.

Each class is **immutable** once constructed. If the class has an associated builder, then it has a `toBuilder()` method, which can be used to convert it back to a builder for making a modified copy.

Because each class is immutable, builder modification will *never* affect already built class instances.

Asynchronous execution



The default client is synchronous. To switch to asynchronous execution, call the `async()` method:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAI0kHttpClient;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionCreateParams;
import java.util.concurrent.CompletableFuture;

// Configures using the `openai.apiKey`, `openai.orgId`, `openai.projectId`,
// `openai.webhookSecret` and `openai.baseUrl` system properties
// Or configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID`, `OPENAI_PROJECT_ID`,
// `OPENAI_WEBHOOK_SECRET` and `OPENAI_BASE_URL` environment variables
OpenAIClient client = OpenAI0kHttpClient.fromEnv();

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .addUserMessage("Say this is a test")
    .model(ChatModel.GPT_5_2)
    .build();
CompletableFuture<ChatCompletion> chatCompletion =
    client.async().chat().completions().create(params);
```

Or create an asynchronous client from the beginning:

```
import com.openai.client.OpenAIClientAsync;
import com.openai.client.okhttp.OpenAI0kHttpClientAsync;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionCreateParams;
import java.util.concurrent.CompletableFuture;

// Configures using the `openai.apiKey`, `openai.orgId`, `openai.projectId`,
// `openai.webhookSecret` and `openai.baseUrl` system properties
// Or configures using the `OPENAI_API_KEY`, `OPENAI_ORG_ID`, `OPENAI_PROJECT_ID`,
// `OPENAI_WEBHOOK_SECRET` and `OPENAI_BASE_URL` environment variables
OpenAIClientAsync client = OpenAI0kHttpClientAsync.fromEnv();

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .addUserMessage("Say this is a test")
    .model(ChatModel.GPT_5_2)
    .build();
CompletableFuture<ChatCompletion> chatCompletion = client.chat().completions().create(params);
```

The asynchronous client supports the same options as the synchronous one, except most methods return `CompletableFuture`s.

Streaming



The SDK defines methods that return response "chunk" streams, where each chunk can be individually processed as soon as it arrives instead of waiting on the full response. Streaming methods generally correspond to [SSE](#) or [JSONL](#) responses.

Some of these methods may have streaming and non-streaming variants, but a streaming method will always have a `Streaming` suffix in its name, even if it doesn't have a non-streaming variant.

These streaming methods return `StreamResponse` for synchronous clients:

```
import com.openai.core.http.StreamResponse;
import com.openai.models.chat.completions.ChatCompletionChunk;

try (StreamResponse<ChatCompletionChunk> streamResponse =
client.chat().completions().createStreaming(params)) {
    streamResponse.stream().forEach(chunk -> {
        System.out.println(chunk);
    });
    System.out.println("No more chunks!");
}
```

Or `AsyncStreamResponse` for asynchronous clients:

```
import com.openai.core.http.AsyncStreamResponse;
import com.openai.models.chat.completions.ChatCompletionChunk;
import java.util.Optional;

client.async().chat().completions().createStreaming(params).subscribe(chunk -> {
    System.out.println(chunk);
});

// If you need to handle errors or completion of the stream
client.async().chat().completions().createStreaming(params).subscribe(new
AsyncStreamResponse.Handler<>() {
    @Override
    public void onNext(ChatCompletionChunk chunk) {
        System.out.println(chunk);
    }

    @Override
    public void onComplete(Optional<Throwable> error) {
        if (error.isPresent()) {
            System.out.println("Something went wrong!");
            throw new RuntimeException(error.get());
        } else {
            System.out.println("No more chunks!");
        }
    }
});

// Or use futures
client.async().chat().completions().createStreaming(params)
    .subscribe(chunk -> {
        System.out.println(chunk);
    })
    .onCompleteFuture();
    .whenComplete((unused, error) -> {
        if (error != null) {
            System.out.println("Something went wrong!");
            throw new RuntimeException(error);
        } else {
            System.out.println("No more chunks!");
        }
    });
```

Async streaming uses a dedicated per-client cached thread pool `Executor` to stream without blocking the current thread. This default is suitable for most purposes.

To use a different `Executor`, configure the subscription using the `executor` parameter:

```
import java.util.concurrent.Executor;
import java.util.concurrent.Executors;

Executor executor = Executors.newFixedThreadPool(4);
client.async().chat().completions().createStreaming(params).subscribe(
    chunk -> System.out.println(chunk), executor
);
```

Or configure the client globally using the `streamHandlerExecutor` method:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import java.util.concurrent.Executors;

OpenAIClient client = OpenAIOkHttpClient.builder()
    .fromEnv()
    .streamHandlerExecutor(Executors.newFixedThreadPool(4))
    .build();
```

Streaming helpers



The SDK provides conveniences for streamed chat completions. A `ChatCompletionAccumulator` can record the stream of chat completion chunks in the response as they are processed and accumulate a `ChatCompletion` object similar to that which would have been returned by the non-streaming API.

For a synchronous response add a `Stream.peek()` call to the stream pipeline to accumulate each chunk:

```
import com.openai.core.http.StreamResponse;
import com.openai.helpers.ChatCompletionAccumulator;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionChunk;

ChatCompletionAccumulator chatCompletionAccumulator = ChatCompletionAccumulator.create();

try (StreamResponse<ChatCompletionChunk> streamResponse =
    client.chat().completions().createStreaming(createParams)) {
    streamResponse.stream()
        .peek(chatCompletionAccumulator::accumulate)
        .flatMap(completion -> completion.choices().stream())
        .flatMap(choice -> choice.delta().content().stream())
        .forEach(System.out::print);
}

ChatCompletion chatCompletion = chatCompletionAccumulator.chatCompletion();
```

For an asynchronous response, add the `ChatCompletionAccumulator` to the `subscribe()` call:

```
import com.openai.helpers.ChatCompletionAccumulator;
import com.openai.models.chat.completions.ChatCompletion;

ChatCompletionAccumulator chatCompletionAccumulator = ChatCompletionAccumulator.create();

client.chat()
    .completions()
    .createStreaming(createParams)
    .subscribe(chunk -> chatCompletionAccumulator.accumulate(chunk).choices().stream()
        .flatMap(choice -> choice.delta().content().stream())
        .forEach(System.out::print))
    .onCompleteFuture()
    .join();

ChatCompletion chatCompletion = chatCompletionAccumulator.chatCompletion();
```

The SDK provides conveniences for streamed responses. A `ResponseAccumulator` can record the stream of response events as they are processed and accumulate a `Response` object similar to that which would have been returned by the non-streaming API.

For a synchronous response add a `Stream.peek()` call to the stream pipeline to accumulate each event:

```
import com.openai.core.http.StreamResponse;
import com.openai.helpers.ResponseAccumulator;
import com.openai.models.responses.Response;
import com.openai.models.responses.ResponseStreamEvent;

ResponseAccumulator responseAccumulator = ResponseAccumulator.create();

try (StreamResponse<ResponseStreamEvent> streamResponse =
    client.responses().createStreaming(createParams)) {
    streamResponse.stream()
        .peek(responseAccumulator::accumulate)
        .flatMap(event -> event.outputTextDelta().stream())
        .forEach(textEvent -> System.out.print(textEvent.delta()));
}

Response response = responseAccumulator.response();
```

For an asynchronous response, add the `ResponseAccumulator` to the `subscribe()` call:

```
import com.openai.helpers.ResponseAccumulator;
import com.openai.models.responses.Response;

ResponseAccumulator responseAccumulator = ResponseAccumulator.create();

client.responses()
    .createStreaming(createParams)
    .subscribe(event -> responseAccumulator.accumulate(event)
        .outputTextDelta().ifPresent(textEvent -> System.out.print(textEvent.delta())))
    .onCompleteFuture()
    .join();

Response response = responseAccumulator.response();
```


Structured outputs with JSON schemas



Open AI [Structured Outputs](#) is a feature that ensures that the model will always generate responses that adhere to a supplied [JSON schema](#).

A JSON schema can be defined by creating a [ResponseFormatJsonSchema](#) and setting it on the input parameters. However, for greater convenience, a JSON schema can instead be derived automatically from the structure of an arbitrary Java class. The JSON content from the response will then be converted automatically to an instance of that Java class. A full, working example of the use of Structured Outputs with arbitrary Java classes can be seen in [StructuredOutputsExample](#).

Java classes can contain fields declared to be instances of other classes and can use collections (see [Defining JSON schema properties](#) for more details):

```
class Person {
    public String name;
    public int birthYear;
}

class Book {
    public String title;
    public Person author;
    public int publicationYear;
}

class BookList {
    public List<Book> books;
}
```

Pass the top-level class—`BookList` in this example—to `responseFormat(Class<T>)` when building the parameters and then access an instance of `BookList` from the generated message content in the response:

```
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletionCreateParams;
import com.openai.models.chat.completions.StructuredChatCompletionCreateParams;

StructuredChatCompletionCreateParams<BookList> params = ChatCompletionCreateParams.builder()
    .addUserMessage("List some famous late twentieth century novels.")
    .model(ChatModel.GPT_5_2)
    .responseFormat(BookList.class)
    .build();

client.chat().completions().create(params).choices().stream()
    .flatMap(choice -> choice.message().content().stream())
    .flatMap(bookList -> bookList.books.stream())
    .forEach(book -> System.out.println(book.title + " by " + book.author.name));
```

You can start building the parameters with an instance of [ChatCompletionCreateParams.Builder](#) or [StructuredChatCompletionCreateParams.Builder](#). If you start with the former (which allows for more compact code) the builder type will change to the latter when

`ChatCompletionCreateParams.Builder.responseFormat(Class<T>)` is called.

If a field in a class is optional and does not require a defined value, you can represent this using the `java.util.Optional` class. It is up to the AI model to decide whether to provide a value for that field or leave it empty.

```
import java.util.Optional;

class Book {
    public String title;
    public Person author;
    public int publicationYear;
    public Optional<String> isbn;
}
```

Generic type information for fields is retained in the class's metadata, but *generic type erasure* applies in other scopes. While, for example, a JSON schema defining an array of books can be derived from the `BookList.books` field with type `List<Book>`, a valid JSON schema cannot be derived from a local variable of that same type, so the following will *not* work:

```
List<Book> books = new ArrayList<>();

StructuredChatCompletionCreateParams<List<Book>> params = ChatCompletionCreateParams.builder()
    .responseFormat(books.getClass())
    // ...
    .build();
```

If an error occurs while converting a JSON response to an instance of a Java class, the error message will include the JSON response to assist in diagnosis. For instance, if the response is truncated, the JSON data will be incomplete and cannot be converted to a class instance. If your JSON response may contain sensitive information, avoid logging it directly, or ensure that you redact any sensitive details from the error message.

Local JSON schema validation



Structured Outputs supports a [subset](#) of the JSON Schema language. Schemas are generated automatically from classes to align with this subset. However, due to the inherent structure of the classes, the generated schema may still violate certain OpenAI schema restrictions, such as exceeding the maximum nesting depth or utilizing unsupported data types.

To facilitate compliance, the method `responseFormat(Class<T>)` performs a validation check on the schema derived from the specified class. This validation ensures that all restrictions are adhered to. If any issues are detected, an exception will be thrown, providing a detailed message outlining the reasons for the validation failure.

- **Local Validation:** The validation process occurs locally, meaning no requests are sent to the remote AI model. If the schema passes local validation, it is likely to pass remote validation as well.
- **Remote Validation:** The remote AI model will conduct its own validation upon receiving the JSON schema in the request.
- **Version Compatibility:** There may be instances where local validation fails while remote validation succeeds. This can occur if the SDK version is outdated compared to the restrictions enforced by the remote AI model.
- **Disabling Local Validation:** If you encounter compatibility issues and wish to bypass local validation, you can disable it by passing `JsonSchemaLocalValidation.NO` to the `responseFormat(Class<T>)`,

`JsonSchemaLocalValidation`) method when building the parameters. (The default value for this parameter is `JsonSchemaLocalValidation.YES`.)

```
import com.openai.core.JsonSchemaLocalValidation;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletionCreateParams;
import com.openai.models.chat.completions.StructuredChatCompletionCreateParams;

StructuredChatCompletionCreateParams<BookList> params = ChatCompletionCreateParams.builder()
    .addUserMessage("List some famous late twentieth century novels.")
    .model(ChatModel.GPT_5_2)
    .responseFormat(BookList.class, JsonSchemaLocalValidation.NO)
    .build();
```

By following these guidelines, you can ensure that your structured outputs conform to the necessary schema requirements and minimize the risk of remote validation errors.

Usage with the Responses API



Structured Outputs are also supported for the Responses API. The usage is the same as described except where the Responses API differs slightly from the Chat Completions API. Pass the top-level class to `text(Class<T>)` when building the parameters and then access an instance of the class from the generated message content in the response.

You can start building the parameters with an instance of `ResponseCreateParams.Builder` or `StructuredResponseCreateParams.Builder`. If you start with the former (which allows for more compact code) the builder type will change to the latter when `ResponseCreateParams.Builder.text(Class<T>)` is called.

For a full example of the usage of *Structured Outputs* with the Responses API, see [ResponsesStructuredOutputsExample](#).

Instead of using `ResponseCreateParams.text(Class<T>)`, you can build a `StructuredResponseTextConfig` and set it on the `ResponseCreateParams` using the `text(StructuredResponseTextConfig)` method. Similar to using `ResponseCreateParams`, you can start with a `ResponseTextConfig.Builder` and its `format(Class<T>)` method will change it to a `StructuredResponseTextConfig.Builder`. This also allows you to set the `verbosity` configuration parameter on the text configuration before adding it to the `ResponseCreateParams`.

For a full example of the usage of *Structured Outputs* with the `ResponseTextConfig` and its `verbosity` parameter, see [ResponsesStructuredOutputsVerbosityExample](#).

Usage with streaming



Structured Outputs can also be used with [Streaming](#) and the Chat Completions API. As responses are returned in "chunks", the full response must first be accumulated to concatenate the JSON strings that can then be converted into instances of the arbitrary Java class. Normal streaming operations can be performed while accumulating the JSON strings.

Use the `ChatCompletionAccumulator` as described in the section on [Streaming helpers](#) to accumulate the JSON strings. Once accumulated, use `ChatCompletionAccumulator.chatCompletion(Class<T>)` to convert the accumulated `ChatCompletion` into a `StructuredChatCompletion`. The `StructuredChatCompletion` can then automatically deserialize the JSON strings into instances of your Java class.

For a full example of the usage of *Structured Outputs* with Streaming and the Chat Completions API, see [StructuredOutputsStreamingExample](#).

With the Responses API, accumulate events while streaming using the [ResponseAccumulator](#). Once accumulated, use `ResponseAccumulator.response(Class<T>)` to convert the accumulated `Response` into a [StructuredResponse](#). The `[ StructuredResponse ]` can then automatically deserialize the JSON strings into instances of your Java class.

For a full example of the usage of *Structured Outputs* with Streaming and the Responses API, see [ResponsesStructuredOutputsStreamingExample](#).

Defining JSON schema properties



When a JSON schema is derived from your Java classes, all properties represented by `public` fields or `public` getter methods are included in the schema by default. Non-`public` fields and getter methods are *not* included by default. You can exclude `public`, or include non-`public` fields or getter methods, by using the `@JsonIgnore` or `@JsonProperty` annotations respectively (see [Annotating classes and JSON schemas](#) for details).

If you do not want to define `public` fields, you can define `private` fields and corresponding `public` getter methods. For example, a `private` field `myValue` with a `public` getter method `getMyValue()` will result in a `"myValue"` property being included in the JSON schema. If you prefer not to use the conventional Java "get" prefix for the name of the getter method, then you *must* annotate the getter method with the `@JsonProperty` annotation and the full method name will be used as the property name. You do not have to define any corresponding setter methods if you do not need them.

Each of your classes *must* define at least one property to be included in the JSON schema. A validation error will occur if any class contains no fields or getter methods from which schema properties can be derived. This may occur if, for example:

- There are no fields or getter methods in the class.
- All fields and getter methods are `public`, but all are annotated with `@JsonIgnore`.
- All fields and getter methods are non-`public`, but none are annotated with `@JsonProperty`.
- A field or getter method is declared with a `Map` type. A `Map` is treated like a separate class with no named properties, so it will result in an empty `"properties"` field in the JSON schema.

Annotating classes and JSON schemas



You can use annotations to add further information to the JSON schema derived from your Java classes, or to control which fields or getter methods will be included in the schema. Details from annotations captured in the JSON schema may be used by the AI model to improve its response. The SDK supports the use of [Jackson Databind](#) annotations.

```

import com.fasterxml.jackson.annotation.JsonClassDescription;
import com.fasterxml.jackson.annotation.JsonIgnore;
import com.fasterxml.jackson.annotation.JsonPropertyDescription;

class Person {
    @JsonPropertyDescription("The first name and surname of the person")
    public String name;
    public int birthYear;
    @JsonPropertyDescription("The year the person died, or 'present' if the person is living.")
    public String deathYear;
}

@JsonClassDescription("The details of one published book")
class Book {
    public String title;
    public Person author;
    @JsonPropertyDescription("The year in which the book was first published.")
    public int publicationYear;
    @JsonIgnore public String genre;
}

class BookList {
    public List<Book> books;
}

```

- Use `@JsonClassDescription` to add a detailed description to a class.
- Use `@JsonPropertyDescription` to add a detailed description to a field or getter method of a class.
- Use `@JsonIgnore` to exclude a `public` field or getter method of a class from the generated JSON schema.
- Use `@JsonProperty` to include a non-`public` field or getter method of a class in the generated JSON schema.

If you use `@JsonProperty(required = false)`, the `false` value will be ignored. OpenAPI JSON schemas must mark all properties as *required*, so the schema generated from your Java classes will respect that restriction and ignore any annotation that would violate it.

You can also use [OpenAPI Swagger 2](#) `@Schema` and `@ArraySchema` annotations. These allow type-specific constraints to be added to your schema properties. You can learn more about the supported constraints in the OpenAPI documentation on [Supported properties](#).

```

import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.media.ArraySchema;

class Article {
    @ArraySchema(minItems = 1, maxItems = 10)
    public List<String> authors;

    @Schema(pattern = "^[A-Za-z ]+$")
    public String title;

    @Schema(format = "date")
    public String publicationDate;

    @Schema(minimum = "1")
    public int pageCount;
}

```

Local validation will check that you have not used any unsupported constraint keywords. However, the values of the constraints are *not* validated locally. For example, if you use a value for the `"format"` constraint of a string property that is not in the list of [supported format names](#), then local validation will pass, but the AI model may report an error.

If you use both Jackson and Swagger annotations to set the same schema field, the Jackson annotation will take precedence. In the following example, the description of `myProperty` will be set to "Jackson description"; "Swagger description" will be ignored:

```
import com.fasterxml.jackson.annotation.JsonPropertyDescription;
import io.swagger.v3.oas.annotations.media.Schema;

class MyObject {
    @Schema(description = "Swagger description")
    @JsonPropertyDescription("Jackson description")
    public String myProperty;
}
```

Function calling with JSON schemas



OpenAI [Function Calling](#) lets you integrate external functions directly into the language model's responses. Instead of producing plain text, the model can output instructions (with parameters) for calling a function when appropriate. You define a [JSON schema](#) for functions, and the model uses it to decide when and how to trigger these calls, enabling more interactive, data-driven applications.

A JSON schema describing a function's parameters can be defined via the API by building a `ChatCompletionTool` containing a `FunctionDefinition` and then using `addTool` to set it on the input parameters. The response from the AI model may then contain requests to call your functions, detailing the functions' names and their parameter values as JSON data that conforms to the JSON schema from the function definition. You can then parse the parameter values from this JSON, invoke your functions, and pass your functions' results back to the AI model. A full, working example of *Function Calling* using the low-level API can be seen in [FunctionCallingRawExample](#).

However, for greater convenience, the SDK can derive a function and its parameters automatically from the structure of an arbitrary Java class: the class's name provides the function name, and the class's fields define the function's parameters. When the AI model responds with the parameter values in JSON form, you can then easily convert that JSON to an instance of your Java class and use the parameter values to invoke your custom function. A full, working example of the use of *Function Calling* with Java classes to define function parameters can be seen in [FunctionCallingExample](#).

Like for [Structured Outputs](#), Java classes can contain fields declared to be instances of other classes and can use collections (see [Defining JSON schema properties](#) for more details). Optionally, annotations can be used to set the descriptions of the function (class) and its parameters (fields) to assist the AI model in understanding the purpose of the function and the possible values of its parameters.

```

import com.fasterxml.jackson.annotation.JsonClassDescription;
import com.fasterxml.jackson.annotation.JsonPropertyDescription;

@JsonClassDescription("Gets the quality of the given SDK.")
static class GetSdkQuality {
    @JsonPropertyDescription("The name of the SDK.")
    public String name;

    public SdkQuality execute() {
        return new SdkQuality(
            name, name.contains("OpenAI") ? "It's robust and polished!" : "*shrug*");
    }
}

static class SdkQuality {
    public String quality;

    public SdkQuality(String name, String evaluation) {
        quality = name + ": " + evaluation;
    }
}

@JsonClassDescription("Gets the review score (out of 10) for the named SDK.")
static class GetSdkScore {
    public String name;

    public int execute() {
        return name.contains("OpenAI") ? 10 : 3;
    }
}

```

When your functions are defined, add them to the input parameters using `addTool(Class<T>)` and then call them if requested to do so in the AI model's response. `Function.arguments(Class<T>)` can be used to parse a function's parameters in JSON form to an instance of your function-defining class. The fields of that instance will be set to the values of the parameters to the function call.

After calling the function, use `ChatCompletionToolMessageParam.Builder.contentAsJson(Object)` to pass the function's result back to the AI model. The method will convert the result to JSON form for consumption by the model. The `Object` can be any object, including simple `String` instances and boxed primitive types.

```

import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.*;
import java.util.Collection;

OpenAIClient client = OpenAIOkHttpClient.fromEnv();

ChatCompletionCreateParams.Builder createParamsBuilder = ChatCompletionCreateParams.builder()
    .model(ChatModel.GPT_3_5_TURBO)
    .maxCompletionTokens(2048)
    .addTool(GetSdkQuality.class)
    .addTool(GetSdkScore.class)
    .addUserMessage("How good are the following SDKs and what do reviewers say: "
        + "OpenAI Java SDK, Unknown Company SDK.");

client.chat().completions().create(createParamsBuilder.build()).choices().stream()
    .map(ChatCompletion.Choice::message)
    // Add each assistant message onto the builder so that we keep track of the
    // conversation for asking a follow-up question later.
    .peek(createParamsBuilder::addMessage)
    .flatMap(message -> {
        message.content().ifPresent(System.out::println);
        return message.toolCalls().stream().flatMap(Collection::stream);
    })
    .forEach(toolCall -> {
        Object result = callFunction(toolCall.function());
        // Add the tool call result to the conversation.
        createParamsBuilder.addMessage(ChatCompletionToolMessageParam.builder()
            .toolCallId(toolCall.id())
            .contentAsJson(result)
            .build());
    });

// Ask a follow-up question about the function call result.
createParamsBuilder.addUserMessage("Why do you say that?");
client.chat().completions().create(createParamsBuilder.build()).choices().stream()
    .flatMap(choice -> choice.message().content().stream())
    .forEach(System.out::println);

static Object callFunction(ChatCompletionMessageToolCall.Function function) {
    switch (function.name()) {
        case "GetSdkQuality":
            return function.arguments(GetSdkQuality.class).execute();
        case "GetSdkScore":
            return function.arguments(GetSdkScore.class).execute();
        default:
            throw new IllegalArgumentException("Unknown function: " + function.name());
    }
}

```

In the code above, an `execute()` method encapsulates each function's logic. However, there is no requirement to follow that pattern. You are free to implement your function's logic in any way that best suits your use case. The pattern above is only intended to *suggest* that a suitable pattern may make the process of function calling simpler to understand and implement.

Usage with the Responses API



Function Calling is also supported for the Responses API. The usage is the same as described except where the Responses API differs slightly from the Chat Completions API. Pass the top-level class to `addTool(Class<T>)` when building the parameters. In the response, look for `ReponseOutputItem` instances that are function calls. Parse the parameters to each function call to an instance of the class using `ResponseFunctionToolCall.arguments(Class<T>)`. Finally, pass the result of each call back to the model.

For a full example of the usage of *Function Calling* with the Responses API using the low-level API to define and parse function parameters, see [ResponsesFunctionCallingRawExample](#).

For a full example of the usage of *Function Calling* with the Responses API using Java classes to define and parse function parameters, see [ResponsesFunctionCallingExample](#).

Local function JSON schema validation



Like for *Structured Outputs*, you can perform local validation to check that the JSON schema derived from your function class respects the restrictions imposed by OpenAI on such schemas. Local validation is enabled by default, but it can be disabled by adding `JsonSchemaLocalValidation.NO` to the call to `addTool`.

```
ChatCompletionCreateParams.Builder createParamsBuilder = ChatCompletionCreateParams.builder()
    .model(ChatModel.GPT_3_5_TURBO)
    .maxCompletionTokens(2048)
    .addTool(GetSdkQuality.class, JsonSchemaLocalValidation.NO)
    .addTool(GetSdkScore.class, JsonSchemaLocalValidation.NO)
    .addUserMessage("How good are the following SDKs and what do reviewers say: "
        + "OpenAI Java SDK, Unknown Company SDK.");
```

See [Local JSON schema validation](#) for more details on local schema validation and under what circumstances you might want to disable it.

Annotating function classes



You can use annotations to add further information about functions to the JSON schemas that are derived from your function classes, or to control which fields or getter methods will be used as parameters to the function. Details from annotations captured in the JSON schema may be used by the AI model to improve its response. The SDK supports the use of [Jackson Databind](#) annotations.

- Use `@JsonClassDescription` to add a description to a function class detailing when and how to use that function.
- Use `@JsonTypeName` to set the function name to something other than the simple name of the class, which is used by default.
- Use `@JsonPropertyDescription` to add a detailed description to function parameter (a field or getter method of a function class).
- Use `@JsonIgnore` to exclude a `public` field or getter method of a class from the generated JSON schema for a function's parameters.
- Use `@JsonProperty` to include a non-`public` field or getter method of a class in the generated JSON schema for a function's parameters.

OpenAI provides some [Best practices for defining functions](#) that may help you to understand how to use the above annotations effectively for your functions.

See also [Defining JSON schema properties](#) for more details on how to use fields and getter methods and combine access modifiers and annotations to define the parameters of your functions. The same rules apply to function classes and to the structured output classes described in that section.

File uploads



The SDK defines methods that accept files.

To upload a file, pass a `Path`:

```
import com.openai.models.files.FileCreateParams;
import com.openai.models.files.FileObject;
import com.openai.models.files.FilePurpose;
import java.nio.file.Paths;

FileCreateParams params = FileCreateParams.builder()
    .purpose(FilePurpose.FINE_TUNE)
    .file(Paths.get("input.jsonl"))
    .build();
FileObject fileObject = client.files().create(params);
```

Or an arbitrary `InputStream`:

```
import com.openai.models.files.FileCreateParams;
import com.openai.models.files.FileObject;
import com.openai.models.files.FilePurpose;
import java.net.URL;

FileCreateParams params = FileCreateParams.builder()
    .purpose(FilePurpose.FINE_TUNE)
    .file(new URL("https://example.com/input.jsonl").openStream())
    .build();
FileObject fileObject = client.files().create(params);
```

Or a `byte[]` array:

```
import com.openai.models.files.FileCreateParams;
import com.openai.models.files.FileObject;
import com.openai.models.files.FilePurpose;

FileCreateParams params = FileCreateParams.builder()
    .purpose(FilePurpose.FINE_TUNE)
    .file("content".getBytes())
    .build();
FileObject fileObject = client.files().create(params);
```

Note that when passing a non-`Path` its filename is unknown so it will not be included in the request. To manually set a filename, pass a `MultipartField`:

```
import com.openai.core.MultipartField;
import com.openai.models.files.FileCreateParams;
import com.openai.models.files.FileObject;
import com.openai.models.files.FilePurpose;
import java.io.InputStream;
import java.net.URL;

FileCreateParams params = FileCreateParams.builder()
    .purpose(FilePurpose.FINE_TUNE)
    .file(MultipartField.<InputStream>builder()
        .value(new URL("https://example.com/input.jsonl").openStream())
        .filename("input.jsonl")
        .build())
    .build();
FileObject fileObject = client.files().create(params);
```

Webhook Verification



Verifying webhook signatures is *optional but encouraged*.

For more information about webhooks, see [the API docs](#).

Parsing webhook payloads



For most use cases, you will likely want to verify the webhook and parse the payload at the same time. To achieve this, we provide the method `client.webhooks().unwrap()`, which parses a webhook request and verifies that it was sent by OpenAI. This method will throw an exception if the signature is invalid.

Note that the `body` parameter must be the raw JSON string sent from the server (do not parse it first). The `.unwrap()` method will parse this JSON for you into an event object after verifying the webhook was sent from OpenAI.

```

import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.core.http.Headers;
import com.openai.models.webhooks.UnwrapWebhookEvent;
import java.util.Optional;

OpenAIClient client = OpenAIOkHttpClient.fromEnv(); // OPENAI_WEBHOOK_SECRET env var used by default

public void handleWebhook(String body, Map<String, String> headers) {
    try {
        Headers headersList = Headers.builder()
            .putAll(headers)
            .build();

        UnwrapWebhookEvent event = client.webhooks().unwrap(body, headersList, Optional.empty());

        if (event.isResponseCompletedWebhookEvent()) {
            System.out.println("Response completed: " +
event.asResponseCompletedWebhookEvent().data());
        } else if (event.isResponseFailed()) {
            System.out.println("Response failed: " + event.asResponseFailed().data());
        } else {
            System.out.println("Unhandled event type: " + event.getClass().getSimpleName());
        }
    } catch (Exception e) {
        System.err.println("Invalid webhook signature: " + e.getMessage());
        // Handle invalid signature
    }
}

```

Verifying webhook payloads directly



In some cases, you may want to verify the webhook separately from parsing the payload. If you prefer to handle these steps separately, we provide the method `client.webhooks().verifySignature()` to *only verify* the signature of a webhook request. Like `.unwrap()`, this method will throw an exception if the signature is invalid.

Note that the `body` parameter must be the raw JSON string sent from the server (do not parse it first). You will then need to parse the body after verifying the signature.

```

import com.fasterxml.jackson.databind.ObjectMapper;
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.core.http.Headers;
import com.openai.models.webhooks.WebhookVerificationParams;
import java.util.Optional;

OpenAIClient client = OpenAIOkHttpClient.fromEnv(); // OPENAI_WEBHOOK_SECRET env var used by
default
ObjectMapper objectMapper = new ObjectMapper();

public void handleWebhook(String body, Map<String, String> headers) {
    try {
        Headers headersList = Headers.builder()
            .putAll(headers)
            .build();

        client.webhooks().verifySignature(
            WebhookVerificationParams.builder()
                .payload(body)
                .headers(headersList)
                .build()
        );

        // Parse the body after verification
        Map<String, Object> event = objectMapper.readValue(body, Map.class);
        System.out.println("Verified event: " + event);
    } catch (Exception e) {
        System.err.println("Invalid webhook signature: " + e.getMessage());
        // Handle invalid signature
    }
}

```

Binary responses



The SDK defines methods that return binary responses, which are used for API responses that shouldn't necessarily be parsed, like non-JSON data.

These methods return [HttpResponse](#):

```

import com.openai.core.http.HttpResponse;
import com.openai.models.files.FileContentParams;

HttpResponse response = client.files().content("file_id");

```

To save the response content to a file, use the [Files.copy\(...\)](#) method:

```
import com.openai.core.http.HttpResponse;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;

try (HttpResponse response = client.files().content(params)) {
    Files.copy(
        response.body(),
        Paths.get(path),
        StandardCopyOption.REPLACE_EXISTING
    );
} catch (Exception e) {
    System.out.println("Something went wrong!");
    throw new RuntimeException(e);
}
```

Or transfer the response content to any `OutputStream`:

```
import com.openai.core.http.HttpResponse;
import java.nio.file.Files;
import java.nio.file.Paths;

try (HttpResponse response = client.files().content(params)) {
    response.body().transferTo(Files.newOutputStream(Paths.get(path)));
} catch (Exception e) {
    System.out.println("Something went wrong!");
    throw new RuntimeException(e);
}
```

Raw responses



The SDK defines methods that deserialize responses into instances of Java classes. However, these methods don't provide access to the response headers, status code, or the raw response body.

To access this data, prefix any HTTP method call on a client or service with `withRawResponse()`:

```
import com.openai.core.http.Headers;
import com.openai.core.http.HttpResponseFor;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .addUserMessage("Say this is a test")
    .model(ChatModel.GPT_5_2)
    .build();
HttpResponseFor<ChatCompletion> chatCompletion =
    client.chat().completions().withRawResponse().create(params);

int statusCode = chatCompletion.statusCode();
Headers headers = chatCompletion.headers();
```

You can still deserialize the response into an instance of a Java class if needed:

```
import com.openai.models.chat.completions.ChatCompletion;

ChatCompletion parsedChatCompletion = chatCompletion.parse();
```

Request IDs



For more information on debugging requests, see [the API docs](#).

When using raw responses, you can access the `x-request-id` response header using the `requestId()` method:

```
import com.openai.core.http.HttpResponseFor;
import com.openai.models.chat.completions.ChatCompletion;
import java.util.Optional;

HttpResponseFor<ChatCompletion> chatCompletion =
    client.chat().completions().withRawResponse().create(params);
Optional<String> requestId = chatCompletion.requestId();
```

This can be used to quickly log failing requests and report them back to OpenAI.

Error handling



The SDK throws custom unchecked exception types:

- `OpenAIServiceException`: Base class for HTTP errors. See this table for which exception subclass is thrown for each HTTP status code:

Status	Exception
400	<code>BadRequestException</code>
401	<code>UnauthorizedException</code>
403	<code>PermissionDeniedException</code>
404	<code>NotFoundException</code>
422	<code>UnprocessableEntityException</code>
429	<code>RateLimitException</code>
5xx	<code>InternalServerErrorException</code>
others	<code>UnexpectedStatusCodeException</code>

`SseException` is thrown for errors encountered during [SSE streaming](#) after a successful initial HTTP response.

- `OpenAIIOException`: I/O networking errors.

- `OpenAIRetryableException` : Generic error indicating a failure that could be retried by the client.
- `OpenAIInvalidDataException` : Failure to interpret successfully parsed data. For example, when accessing a property that's supposed to be required, but the API unexpectedly omitted it from the response.
- `OpenAIException` : Base class for all exceptions. Most errors will result in one of the previously mentioned ones, but completely generic errors may be thrown using the base class.

Pagination



The SDK defines methods that return a paginated lists of results. It provides convenient ways to access the results either one page at a time or item-by-item across all pages.

Auto-pagination



To iterate through all results across all pages, use the `autoPager()` method, which automatically fetches more pages as needed.

When using the synchronous client, the method returns an `Iterable`

```
import com.openai.models.finetuning.jobs.FineTuningJob;
import com.openai.models.finetuning.jobs.JobListPage;

JobListPage page = client.fineTuning().jobs().list();

// Process as an Iterable
for (FineTuningJob job : page.autoPager()) {
    System.out.println(job);
}

// Process as a Stream
page.autoPager()
    .stream()
    .limit(50)
    .forEach(job -> System.out.println(job));
```

When using the asynchronous client, the method returns an `AsyncStreamResponse` :


```

import com.openai.core.http.AsyncStreamResponse;
import com.openai.models.finetuning.jobs.FineTuningJob;
import com.openai.models.finetuning.jobs.JobListPageAsync;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;

CompletableFuture<JobListPageAsync> pageFuture = client.async().fineTuning().jobs().list();

pageFuture.thenRun(page -> page.autoPager().subscribe(job -> {
    System.out.println(job);
}));

// If you need to handle errors or completion of the stream
pageFuture.thenRun(page -> page.autoPager().subscribe(new AsyncStreamResponse.Handler<>() {
    @Override
    public void onNext(FineTuningJob job) {
        System.out.println(job);
    }

    @Override
    public void onComplete(Optional<Throwable> error) {
        if (error.isPresent()) {
            System.out.println("Something went wrong!");
            throw new RuntimeException(error.get());
        } else {
            System.out.println("No more!");
        }
    }
}));

// Or use futures
pageFuture.thenRun(page -> page.autoPager()
    .subscribe(job -> {
        System.out.println(job);
    })
    .onCompleteFuture()
    .whenComplete((unused, error) -> {
        if (error != null) {
            System.out.println("Something went wrong!");
            throw new RuntimeException(error);
        } else {
            System.out.println("No more!");
        }
    })
    ));

```

Manual pagination



To access individual page items and manually request the next page, use the `items()`, `hasNextPage()`, and `nextPage()` methods:

```
import com.openai.models.finetuning.jobs.FineTuningJob;
import com.openai.models.finetuning.jobs.JobListPage;

JobListPage page = client.fineTuning().jobs().list();
while (true) {
    for (FineTuningJob job : page.items()) {
        System.out.println(job);
    }

    if (!page.hasNextPage()) {
        break;
    }

    page = page.nextPage();
}
```

Logging



The SDK uses the standard [OkHttp logging interceptor](#).

Enable logging by setting the `OPENAI_LOG` environment variable to `info`:

```
export OPENAI_LOG=info
```

Or to `debug` for more verbose logging:

```
export OPENAI_LOG=debug
```

ProGuard and R8



Although the SDK uses reflection, it is still usable with [ProGuard](#) and [R8](#) because `openai-java-core` is published with a [configuration file](#) containing [keep rules](#).

ProGuard and R8 should automatically detect and use the published rules, but you can also manually copy the keep rules if necessary.

GraalVM



Although the SDK uses reflection, it is still usable in [GraalVM](#) because `openai-java-core` is published with [reachability metadata](#).

GraalVM should automatically detect and use the published metadata, but [manual configuration](#) is also available.

Spring Boot



If you're using Spring Boot, then you can use the SDK's [Spring Boot starter](#) to simplify configuration and get set up quickly.

Installation



Gradle



```
implementation("com.openai:openai-java-spring-boot-starter:4.30.0")
```

Maven



```
<dependency>
  <groupId>com.openai</groupId>
  <artifactId>openai-java-spring-boot-starter</artifactId>
  <version>4.30.0</version>
</dependency>
```

Configuration



The [client's environment variable options](#) can be configured in `application.properties` or `application.yml`.

`application.properties`



```
openai.base-url=https://api.openai.com/v1
openai.api-key=My API Key
openai.org-id=My Organization
openai.project-id=My Project
openai.webhook-secret=My Webhook Secret
```

`application.yml`



```
openai:
  base-url: https://api.openai.com/v1
  api-key: My API Key
  org-id: My Organization
  project-id: My Project
  webhook-secret: My Webhook Secret
```

Other configuration



Configure any other client option by providing one or more instances of `OpenAIClientCustomizer`. For example, here's how you'd set `maxRetries`:

```
import com.openai.springboot.OpenAIClientCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenAIConfig {
    @Bean
    public OpenAIClientCustomizer customizer() {
        return builder -> builder.maxRetries(3);
    }
}
```

Usage



Inject `OpenAIClient` anywhere and start using it!

Jackson



The SDK depends on [Jackson](#) for JSON serialization/deserialization. It is compatible with version 2.13.4 or higher, but depends on version 2.18.2 by default.

The SDK throws an exception if it detects an incompatible Jackson version at runtime (e.g. if the default version was overridden in your Maven or Gradle config).

If the SDK threw an exception, but you're *certain* the version is compatible, then disable the version check using the `checkJacksonVersionCompatibility` on `OpenAI0kHttpClient` or `OpenAI0kHttpClientAsync`.

⚠Caution

We make no guarantee that the SDK works correctly when the Jackson version check is disabled.

Also note that there are bugs in older Jackson versions that can affect the SDK. We don't work around all Jackson bugs ([example](#)) and expect users to upgrade Jackson for those instead.

Microsoft Azure



To use this library with [Azure OpenAI](#), use the same OpenAI client builder but with the Azure-specific configuration.

```

OpenAIClient client = OpenAIOkHttpClient.builder()
    // Gets the API key and endpoint from the `AZURE_OPENAI_KEY` and `OPENAI_BASE_URL`
    environment variables, respectively
    .fromEnv()
    // Set the Azure Entra ID
    .credential(BearerTokenCredential.create(AuthenticationUtil.getBearerTokenSupplier(
        new DefaultAzureCredentialBuilder().build(),
        "https://cognitiveservices.azure.com/.default")))
    .build();

```

See the complete Azure OpenAI example in the [openai-java-example](#) directory. The other examples in the directory also work with Azure as long as the client is configured to use it.

Optional: URL path mode configuration



The [ClientOptions](#) can be configured to treat Azure OpenAI endpoint URLs differently, depending on your service setup. The default value is [AzureUrlPathMode.AUTO](#). To customize the SDK behavior, each value does the following:

- [AzureUrlPathMode.LEGACY](#) : forces the deployment or model name into the path.
- [AzureUrlPathMode.UNIFIED](#) : for newer endpoints ending in `/openai/v1` the service behaviour matches OpenAI's, therefore [AzureOpenAIServiceVersion](#) becomes optional and the model is passed in the request object.
- [AzureUrlPathMode.AUTO](#) : automatically detects the path mode based on the base URL. Default value.

Network options



Retries



The SDK automatically retries 2 times by default, with a short exponential backoff between requests.

Only the following error types are retried:

- Connection errors (for example, due to a network connectivity problem)
- 408 Request Timeout
- 409 Conflict
- 429 Rate Limit
- 5xx Internal

The API may also explicitly instruct the SDK to retry or not retry a request.

To set a custom number of retries, configure the client using the [maxRetries](#) method:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAI0kHttpClient;

OpenAIClient client = OpenAI0kHttpClient.builder()
    .fromEnv()
    .maxRetries(4)
    .build();
```

Timeouts



Requests time out after 10 minutes by default.

To set a custom timeout, configure the method call using the `timeout` method:

```
import com.openai.models.chat.completions.ChatCompletion;

ChatCompletion chatCompletion = client.chat().completions().create(
    params, RequestOptions.builder().timeout(Duration.ofSeconds(30)).build()
);
```

Or configure the default for all method calls at the client level:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAI0kHttpClient;
import java.time.Duration;

OpenAIClient client = OpenAI0kHttpClient.builder()
    .fromEnv()
    .timeout(Duration.ofSeconds(30))
    .build();
```

Proxies



To route requests through a proxy, configure the client using the `proxy` method:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAI0kHttpClient;
import java.net.InetSocketAddress;
import java.net.Proxy;

OpenAIClient client = OpenAI0kHttpClient.builder()
    .fromEnv()
    .proxy(new Proxy(
        Proxy.Type.HTTP, new InetSocketAddress(
            "https://example.com", 8080
        )
    ))
    .build();
```

Connection pooling



To customize the underlying OkHttp connection pool, configure the client using the `maxIdleConnections` and `keepAliveDuration` methods:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import java.time.Duration;

OpenAIClient client = OpenAIOkHttpClient.builder()
    .fromEnv()
    // If `maxIdleConnections` is set, then `keepAliveDuration` must be set, and vice versa.
    .maxIdleConnections(10)
    .keepAliveDuration(Duration.ofMinutes(2))
    .build();
```

If both options are unset, OkHttp's default connection pool settings are used.

HTTPS



Note

Most applications should not call these methods, and instead use the system defaults. The defaults include special optimizations that can be lost if the implementations are modified.

To configure how HTTPS connections are secured, configure the client using the `sslSocketFactory`, `trustManager`, and `hostnameVerifier` methods:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;

OpenAIClient client = OpenAIOkHttpClient.builder()
    .fromEnv()
    // If `sslSocketFactory` is set, then `trustManager` must be set, and vice versa.
    .sslSocketFactory(yourSSLSocketFactory)
    .trustManager(yourTrustManager)
    .hostnameVerifier(yourHostnameVerifier)
    .build();
```

Custom HTTP client



The SDK consists of three artifacts:

- `openai-java-core`
 - Contains core SDK logic
 - Does not depend on `OkHttp`
 - Exposes `OpenAIClient`, `OpenAIClientAsync`, `OpenAIClientImpl`, and `OpenAIClientAsyncImpl`, all of which can work with any HTTP client
- `openai-java-client-okhttp`
 - Depends on `OkHttp`

- Exposes `OpenAI0kHttpClient` and `OpenAI0kHttpClientAsync`, which provide a way to construct `OpenAIClientImpl` and `OpenAIClientAsyncImpl`, respectively, using `OkHttp`
- `openai-java`
 - Depends on and exposes the APIs of both `openai-java-core` and `openai-java-client-okhttp`
 - Does not have its own logic

This structure allows replacing the SDK's default HTTP client without pulling in unnecessary dependencies.

Customized `OkHttpClient`



Try the available [network options](#) before replacing the default client.

To use a customized `OkHttpClient` :

1. Replace your `openai-java` dependency with `openai-java-core`
2. Copy `openai-java-client-okhttp`'s `OkHttpClient` class into your code and customize it
3. Construct `OpenAIClientImpl` or `OpenAIClientAsyncImpl`, similarly to `OpenAI0kHttpClient` or `OpenAI0kHttpClientAsync`, using your customized client

Completely custom HTTP client



To use a completely custom HTTP client:

1. Replace your `openai-java` dependency with `openai-java-core`
2. Write a class that implements the `HttpClient` interface
3. Construct `OpenAIClientImpl` or `OpenAIClientAsyncImpl`, similarly to `OpenAI0kHttpClient` or `OpenAI0kHttpClientAsync`, using your new client class

Undocumented API functionality



The SDK is typed for convenient usage of the documented API. However, it also supports working with undocumented or not yet supported parts of the API.

Parameters



To set undocumented parameters, call the `putAdditionalHeader`, `putAdditionalQueryParam`, or `putAdditionalBodyProperty` methods on any `Params` class:


```
import com.openai.core.JsonValue;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .putAdditionalHeader("Secret-Header", "42")
    .putAdditionalQueryParam("secret_query_param", "42")
    .putAdditionalBodyProperty("secretProperty", JsonValue.from("42"))
    .build();
```

These can be accessed on the built object later using the `_additionalHeaders()`, `_additionalQueryParams()`, and `_additionalBodyProperties()` methods.

To set undocumented parameters on *nested* headers, query params, or body classes, call the `putAdditionalProperty` method on the nested class:

```
import com.openai.core.JsonValue;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .responseFormat(ChatCompletionCreateParams.ResponseFormat.builder()
        .putAdditionalProperty("secretProperty", JsonValue.from("42"))
        .build())
    .build();
```

These properties can be accessed on the nested built object later using the `_additionalProperties()` method.

To set a documented parameter or property to an undocumented or not yet supported *value*, pass a `JsonValue` object to its setter:

```
import com.openai.core.JsonValue;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .messages(JsonValue.from(42))
    .model(ChatModel.GPT_5_2)
    .build();
```

The most straightforward way to create a `JsonValue` is using its `from(...)` method:

```

import com.openai.core.JsonValue;
import java.util.List;
import java.util.Map;

// Create primitive JSON values
JsonValue nullValue = JsonValue.from(null);
JsonValue booleanValue = JsonValue.from(true);
JsonValue numberValue = JsonValue.from(42);
JsonValue stringValue = JsonValue.from("Hello World!");

// Create a JSON array value equivalent to `["Hello", "World"]`
JsonValue arrayValue = JsonValue.from(List.of(
    "Hello", "World"
));

// Create a JSON object value equivalent to `{ "a": 1, "b": 2 }`
JsonValue objectValue = JsonValue.from(Map.of(
    "a", 1,
    "b", 2
));

// Create an arbitrarily nested JSON equivalent to:
// {
//   "a": [1, 2],
//   "b": [3, 4]
// }
JsonValue complexValue = JsonValue.from(Map.of(
    "a", List.of(
        1, 2
    ),
    "b", List.of(
        3, 4
    )
));

```

Normally a `Builder` class's `build` method will throw `IllegalStateException` if any required parameter or property is unset.

To forcibly omit a required parameter or property, pass `JsonMissing`:

```

import com.openai.core.JsonMissing;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
    .model(ChatModel.GPT_5_4)
    .messages(JsonMissing.of())
    .build();

```

Response properties



To access undocumented response properties, call the `_additionalProperties()` method:

```
import com.openai.core.JsonValue;
import java.util.Map;

Map<String, JsonValue> additionalProperties =
client.chat().completions().create(params)._additionalProperties();
JsonValue secretPropertyValue = additionalProperties.get("secretProperty");

String result = secretPropertyValue.accept(new JsonValue.Visitor<>() {
    @Override
    public String visitNull() {
        return "It's null!";
    }

    @Override
    public String visitBoolean(boolean value) {
        return "It's a boolean!";
    }

    @Override
    public String visitNumber(Number value) {
        return "It's a number!";
    }

    // Other methods include `visitMissing`, `visitString`, `visitArray`, and `visitObject`
    // The default implementation of each unimplemented method delegates to `visitDefault`, which
    // throws by default, but can also be overridden
});
```

To access a property's raw JSON value, which may be undocumented, call its `_` prefixed method:

```
import com.openai.core.JsonField;
import com.openai.models.chat.completions.ChatCompletionMessageParam;
import java.util.Optional;

JsonField<List<ChatCompletionMessageParam>> messages =
client.chat().completions().create(params)._messages();

if (messages.isMissing()) {
    // The property is absent from the JSON response
} else if (messages.isNull()) {
    // The property was set to literal null
} else {
    // Check if value was provided as a string
    // Other methods include `asNumber()`, `asBoolean()`, etc.
    Optional<String> jsonString = messages.asString();

    // Try to deserialize into a custom type
    MyClass myObject = messages.asUnknown().orElseThrow().convert(MyClass.class);
}
```

Response validation



In rare cases, the API may return a response that doesn't match the expected type. For example, the SDK may expect a property to contain a `String`, but the API could return something else.

By default, the SDK will not throw an exception in this case. It will throw `OpenAIInvalidDataException` only if you directly access the property.

If you would prefer to check that the response is completely well-typed upfront, then either call `validate()`:

```
import com.openai.models.chat.completions.ChatCompletion;

ChatCompletion chatCompletion = client.chat().completions().create(params).validate();
```

Or configure the method call to validate the response using the `responseValidation` method:

```
import com.openai.models.chat.completions.ChatCompletion;

ChatCompletion chatCompletion = client.chat().completions().create(
    params, RequestOptions.builder().responseValidation(true).build()
);
```

Or configure the default for all method calls at the client level:

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAI0kHttpClient;

OpenAIClient client = OpenAI0kHttpClient.builder()
    .fromEnv()
    .responseValidation(true)
    .build();
```

FAQ



Why don't you use plain `enum` classes?



Java `enum` classes are not trivially [forwards compatible](#). Using them in the SDK could cause runtime exceptions if the API is updated to respond with a new enum value.

Why do you represent fields using `JsonField<T>` instead of just plain `T`?



Using `JsonField<T>` enables a few features:

- Allowing usage of [undocumented API functionality](#)
- Lazily [validating the API response against the expected shape](#)
- Representing absent vs explicitly null values

Why don't you use `data` classes?



It is not [backwards compatible to add new fields to a data class](#) and we don't want to introduce a breaking change every time we add a field to a class.

Why don't you use checked exceptions?



Checked exceptions are widely considered a mistake in the Java programming language. In fact, they were omitted from Kotlin for this reason.

Checked exceptions:

- Are verbose to handle
- Encourage error handling at the wrong level of abstraction, where nothing can be done about the error
- Are tedious to propagate due to the [function coloring problem](#)
- Don't play well with lambdas (also due to the function coloring problem)

Semantic versioning



This package generally follows [SemVer](#) conventions, though certain backwards-incompatible changes may be released as minor versions:

1. Changes to library internals which are technically public but not intended or documented for external use. *(Please open a GitHub issue to let us know if you are relying on such internals.)*
2. Changes that we do not expect to impact the vast majority of users in practice.

We take backwards-compatibility seriously and work hard to ensure you can rely on a smooth upgrade experience.

We are keen for your feedback; please open an [issue](#) with questions, bugs, or suggestions.